Gerência de Memória: Segmentação

Professor: Paulo Roberto de Oliveira
Disciplina: Sistemas Operacionais

Departamento de Informática
Universidade Estadual de Maringá

Introdução

- Considere o exemplo de um compilador cuja memória é dividida em 5 áreas:

  - Texto fonte: O texto fonte sendo salvo para a listagem impressa (em sistemas de lote) ;
  - Tabela de símbolos: Nomes e atributos das variáveis;
  - Constantes: Dos tipos inteiro e ponto flutuante;
  - Árvore de parse: Contém a análise sintática do programa;
  - Pilha: Usada para as chamadas de procedimento dentro do compilador.

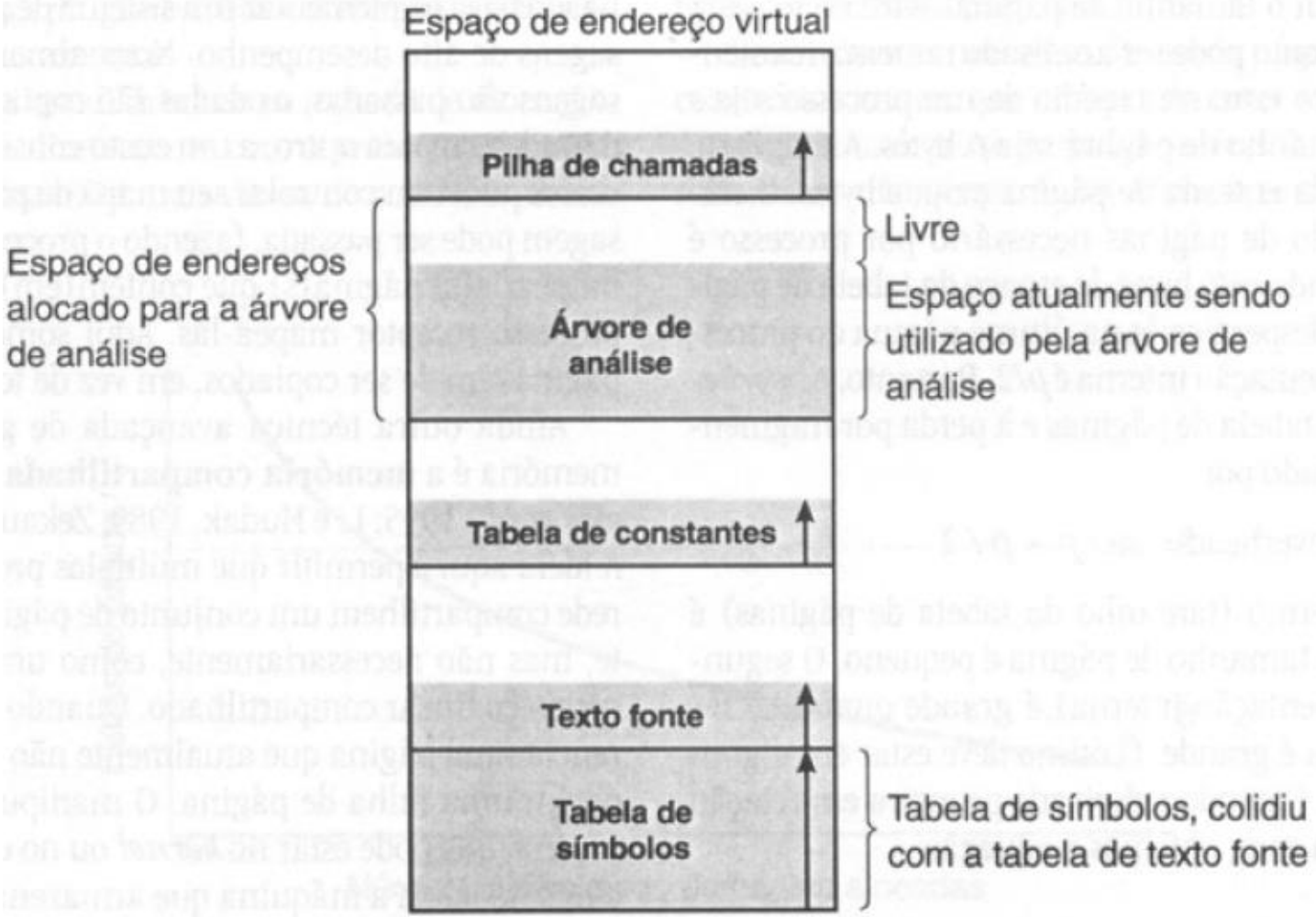
Introdução

- Analisando essas áreas:
  - As 4 primeiras crescem continuamente durante a compilação;
  - A última cresce e diminui dependendo da atividade do programa.

- Num espaço unidimensional, como o espaço virtual, as 5 áreas precisam ser alocadas de forma contínua.

- Prevendo o crescimento, é dado uma margem para cada área crescer. Considere que o programa compilado tem um número muito grande de variáveis declaradas.
  - A tabela de símbolos cresce até ocupar a área reservada, não podendo crescer mais. Porém, há espaço nas demais!

Introdução


Espaço de endereço virtual
Pilha de chamadas
Livre
Espaço de endereços alocado para a árvore de análise
Árvore de análise
Espaço atualmente sendo utilizado pela árvore de análise
Tabela de constantes
Texto fonte
Tabela de símbolos
Tabela de símbolos, colidiu com a tabela de texto fonte

Introdução

- Uma solução é reorganizar a ocupação de todas as áreas do programa - semelhante ao sistema de overlays, em que o programador é o responsável!

- O que é preciso é uma forma automática de controlar o crescimento de tabelas, como a memória virtual libera o programador de controlar os overlays.

- Solução: **Segmentação**.

Segmento

- Sequência linear de endereços;
- Inicia em zero e vai até um máximo;
- Diferentes segmentos podem ter (e, em geral, tem) tamanhos diferentes;
- O tamanho de um segmento pode variar durante a execução (para mais ou para menos);
- O endereçamento deve ser feito em 2 partes:
  - O endereço do segmento;
  - O deslocamento dentro do segmento.
- Segmentos contém entidades de tipos diferentes (procedimentos, tabelas, pilha, buffers, constantes, etc.).

Vantagens da Segmentação

- Se os procedimentos de um programa forem colocados em segmentos distintos, a linkedição é facilitada - o endereço de uma chamada passa a ser (n, end), onde n é o número do segmento e end é o endereço de entrada do procedimento;

- Se um procedimento é alterado, a compilação não altera nenhum dos outros:
  - Mesmo que o programa cresça, os endereços de entrada de todos os procedimentos continuam os mesmos.

Vantagens da Segmentação

- O compartilhamento de procedimentos e bibliotecas (DLLs) é mais simples (basta compartilhar o segmento);

- Já que os segmentos contém elementos de tipos diferentes, a proteção pode ser diferente para cada um;

Vantagens da Segmentação

- É mais fácil compartilhar dados ou procedimentos entre vários processos:
  - Ex.: Bibliotecas grandes, como sistemas gráficos;
  - Em geral, são compiladas e colocadas em todos os programas que necessitam delas;
  - Com segmentação, a biblioteca pode ser colocada em um segmento separado e pode ser compartilhada entre todos – não precisa ser duplicada em cada um;

Paginação X Segmentação

Consideração	Paginação	Segmentação
O total de espaço de endereço pode exceder ao tamanho da memória física.	Sim	Sim
Os procedimentos e os dados podem ser diferenciados e separadamente protegidos.	Não	Sim
As tabelas cujo tamanho varia podem ser acomodadas facilmente.	Não	Sim
O compartilhamento entre usuários é facilitado.	Não	Sim
Porque esta técnica foi inventada.	Para adquirir um grande espaço de endereço sem precisa adquirir mais memória física.	Para permitir que programas e dados sejam divididos em espaços de endereços logicamente independentes e ajudar no compartilhamento e proteção.

Paginação X Segmentação

- Na paginação o conteúdo de uma página é “acidental”.

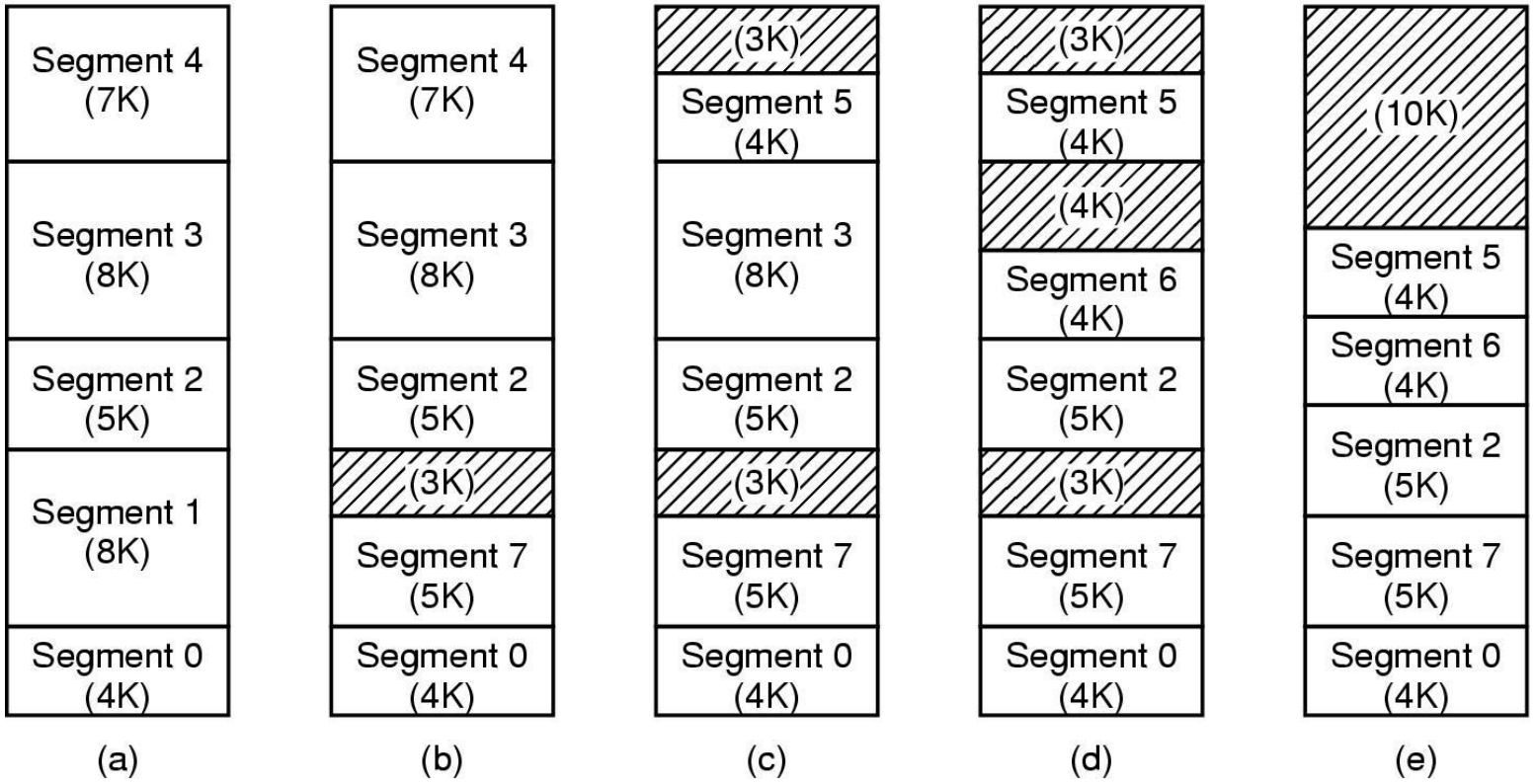
- A proteção individual das páginas é possível:
  - Pode-se colocar alguns bits extras em cada página para controlar a proteção;
  - O programador precisa saber onde ficam as fronteiras para determinar o que fica em cada página;

Implementando Segmentação Pura

- Considerações:
  - Segmentos, como páginas, podem sofrer swap;
  - O espaço de um segmento maior pode ser ocupado por um menor;
  - Isso leva a perda de espaço útil - fragmentação!
  - Uma solução é a compactação, feita pelo SO.

Implementando Segmentação Pura

- Na figura, vemos em *a-d* operações que produzem fragmentação externa. Em *e* vemos a compactação do ambiente.



Considerações Finais

- Usos associados a uma semântica (diferentemente de páginas).
- Seu tamanho variável implica numa estrutura mais complexa para seu uso.
- Em geral, segmentação é usada junto com paginação.
- Possibilita um mecanismo “alto-nível” para o usuário, junto com o mecanismo mais automático de paginação.

Dúvidas?